

Course Code:CS2005 Name of the Programme: B.Tech(Hons.) Name of the Course: Agile Software Engineering and DevOps Semester: IV		
Course credit	No. of hours per week	Total no. of Teaching hours
03	2+0+2	60
Course Objectives:		
Introduce the principles of Agile software development and compare them with traditional software development models.		
Explore various Agile methodologies such as Scrum, XP, Lean, and Kanban, and provide guidelines based on project needs.		
Develop the skills needed to gather software requirements, design specifications, test plans, and coverage analysis in Agile-based projects.		
Equip students with hands-on experience in using modern DevOps tools (e.g., Git, GitHub, Docker) for automation, and deployment in Agile environments.		

Syllabus

Module - 1 Introduction to Agile	No. of hours
	5
Introduction to Software engineering (SDLC), Software process models- Waterfall, V model, Iterative model, Spiral model; Introduction to Agile, Agile Manifesto, Agile development approach, traditional development models, benefits and pitfalls of Agile development, Introduction to OOAD and UML.	

Module - 2 Agile Methodologies and Practices	No. of hours
	5
Introduction to various Agile methodologies such as Scrum, XP, Lean, and Kanban, Scrum Roles - Product Owner, Scrum Master, Team, Release Manager, Project Manager, Product Manager, Architect, events, and artifacts; Product Inception: Product vision, stakeholders, initial backlog creation; Agile Requirements – User personas, story mapping, user stories, 3Cs, INVEST, Acceptance criteria, sprints, requirements, product backlog and backlog grooming; Test First Development; Pair Programming and Code reviews; code coverage and tools.	

Module - 3 Sprint Management and Planning Techniques	No. of hours
	5
Sprint Planning, Sprint Reviews, Sprint Retrospectives Sprint Planning - Agile release and iteration (sprint) planning, Develop Epics and Stories, Estimating Stories, Prioritizing Stories (WSJF technique from SAFe), Iterations/Sprints Overview. Velocity Determination, Iteration Planning Meeting, Iteration, Planning Guidelines, Development, Testing, Daily Stand-up Meetings, Progress Tracking, Velocity Tracking, Monitoring and Controlling: Burn down Charts, Inspect & Adapt (Fishbone Model), Agile Release Train.	

Module - 4 DevOps Foundations	No. of hours
	10
Overview of Devops and why it is needed; Continuous Integration, Build, Testing, Delivery and Deployment(CI and CD): Jenkins, Pylint, PyUnit. Overview of version control, branching and release using GIT;Git/Github/GitActions, Creating pipelines, Setting up runners Containers and container orchestration (Docker, DockerHub and Kubernetes) for application development and deployment.	

Module - 5 Agile testing and Automation	No. of hours
	5
Testing: Functionality Testing, UI Testing(Junit, Sonar), Performance Testing, Security Testing, Load Testing, Stress Testing, A/B testing; Agile Testing: Principles of agile testers; The agile testing quadrants, Agile automation, Test automation pyramid; Test Automation Tools: Selenium, RPA(UiPath), Traceability matrix.	

Course Outcomes: After completing the course, the students will be able to: -

CO 1	Evaluate the advantages and disadvantages of Agile development compared to traditional models
CO 2	Assess various Agile methodologies such as Scrum, XP, Lean, and Kanban, and determine their appropriate applications
CO 3	Create software requirements, design specifications, test plan and Analyze test coverage, requirements traceability for a software project
CO 4	Utilize and implement various DevOps tools (e.g., Git, GitHub, Docker) in a software project
CO 5	Develop a mini software project using Agile Scrum methodology, simulating its roles, meetings, processes, and artifacts

Assessment Methodologies

(Tick✓ the Relevant Methodologies)

Assignments✓	Closed book tests✓	Open book tests
Case study	Student Presentation✓	Mini projects / Model Building✓
MOOC ✓	Quiz✓	Peer Review✓

Textbooks (With ISBN No)

Text Books	
1.	Essential Scrum: A Practical Guide to the Most Popular Agile Process Kenneth S.Rubin 2012,published by Addison-Wesley Professional, ISBN- 978-9332546172
2.	Software Engineering: A Practitioner's Approach By Roger S. Pressman and Bruce MaximMcGraw-Hill Higher International; ISBN-10: 1259872971; ISBN-13: 978-1259872976, 9 th Edition(09/19)
3.	SUCCEEDING WITH AGILE: SOFTWARE DEVELOPMENT USING SCRUM Paperback – 1 January 2015, by Cohn, Pearson Education India, 978-9332547964
4.	The DevOps Handbook: How to Create World-Class Agility, Reliability, and Security in Technology Organizations by Gene Kin, Patrick Debois, John Willis, Jez Humble, and John Allspaw IT Revolution Press; ISBN-10: 1942788002; ISBN-13: 978-1942788003 (10/16)

Reference Material (With ISBN No)

Reference Books	
1.	Software Engineering (10th Edition) by Ian Sommerville Pearson; ISBN-13: 978-0133943030 (04/15)
2.	Agile Software Development: The Cooperative Game Alistair Cockburn 2nd Edition, 2006, Addison-Wesley Professional, ISBN - 978-0321482754
3.	Scrum and XP from the Trenches Henrik Kniberg 2nd Edition, 2015, Published by C4Media, publisher of InfoQ.com. ISBN - 9781329224278

4.	Agile Project Management: Creating Innovative Products, Second Edition By Jim Highsmith, Addison-Wesley Professional, 2009, ISBN- 978-0321658395
5.	Agile Project Management: Managing for Success, By James A. Crowder, Shelli Friess, Springer 2014, ISBN-978-3319090177
6.	Learning Agile: Understanding Scrum, XP, Lean, and Kanban, By Andrew Stellman, Jennifer Greene, 2015, O Reilly, ISBN-9781449331924 DevOps: Continuous Delivery, Integration, and Deployment with DevOps: By SricharanVadapalli, Packt, 2018", ISBN-978-1789132991

Laboratory Component (30 Hours)

SI.	AGILE EXERCISE	MARKS	DUE
1	To understand the Waterfall model and its phases, and to analyze its advantages and disadvantages	10	Week 1
2	To understand the Agile Scrum methodology and its core principles, and to practice implementing it	10	Week 2
3	Identify functional and non-functional requirements of a real-time software project	10	Week 3
4	To understand the importance of software requirements specification and to practice creating a software requirements specification document	10	Week 4
5	To understand the importance of system modeling and to practice creating a system model using UML	10	Week 5
6	Write 2-5 test cases each testing category for a software project: 1) Sanity Testing 2) Security Testing 3) Stress testing 4) Performance Testing	10	Week 6
7	Document the test plan, define list of test cases and traceability matrix of your respective team project using standard IEEE templates	10	Week 7

8	Identify your project's scrum team, Define Product Backlog, High Level Sprint Plan (consisting of max of 6 sprints of duration 1 week each)	10	Week 8
9	Identify your project's scrum team, Define Product Backlog, High Level Sprint Plan (consisting of max of 6 sprints of duration 1 week each)	10	Week 9
10	Live simulation of Daily Scrum, Sprint Planning, Sprint Review, Sprint Retrospective meetings	10	Week 10
11	Hands on lab on Git, Github, and GitAction	10	Week 11
12	Hands on lab on Docker & Docker Hub	10	Week 12
13	To understand the importance of testing in software engineering and to practice testing a software project using PyUnit	10	Week 13
14	Create Agile Testing Quadrants for a real-time software project by putting 2-5 tests in each quadrant	10	Week 14
15	For Live team project do the end-to-end DevOps demo, prepare project report	10	Week 15
	Total Marks: 150/15 = 10 Marks		

Lesson plan (Template –2)

Name of the Programme:	B.Tech(Hons.)	
Title of the Course:	Agile Software Engineering and DevOps	
Course code	CS2005	
Semester:	IV	
Names of the Course Instructor(s)	Prof.Manjul Krishna Gupta, Prof.Deepak Murthy, Prof.Shanthala, Prof.CVSN Reddy, Prof.Sunil Kumar, Prof.Manasa , Prof.Hari Kumar VPS	
Course credit	No. of hours per week (2+0+2)	Total no. of Teaching hours
04	04 Hours	60 Hours

Session	Module No.	Topic	Pedagogy /activities	Pre-reading/reference
1	1	Intro to Software Engineering: SDLC, Waterfall, and V-models	Interactive lecture + Whiteboard	Textbook1: Chapter 3
2	1	Intro to Agile: Agile Manifesto, Principles, and Iterative Models	Interactive lecture + Whiteboard	Textbook1: Chapter 1
3	1	Lab 1: Waterfall model analysis (advantages/disadvantages)	Lab	Textbook1: Chapter 3
4	1	Lab 1: Waterfall model analysis (advantages/disadvantages)	Lab	Textbook1: Chapter 3
5	2	Intro to Scrum Methodology and Core Principles	Interactive lecture + Whiteboard	Textbook1: Chapter 2
6	2	Scrum Framework: Roles, Activities, and Artifacts	Interactive lecture + Whiteboard	Textbook1: Chapter 2
7	2	Lab 2: Agile Scrum methodology implementation	Lab	Textbook1: Chapter 2
8	2	Lab 2: Agile Scrum methodology implementation	Lab	Textbook1: Chapter 2
9	1	Managing Variability and Uncertainty;	Interactive lecture + Whiteboard	Textbook1: Chapter 3
10	2	Agile Requirements: User Personas and Story Mapping	Interactive lecture + Whiteboard	Textbook1: Chapter 5

11	2	Lab 3: Functional and non-functional requirements	Lab	Textbook1: Chapter 5
12	2	Lab 3: Functional and non-functional requirements	Lab	Textbook1: Chapter 5
13	2	Writing Good User Stories (INVEST Criteria) and 3Cs	Interactive lecture + Whiteboard	Textbook1: Chapter 5
14	2	Product Backlog Grooming, Refinement, and Ready Criteria	Interactive lecture + Whiteboard	Textbook1: Chapter 6
15	2	Lab 4: Software Requirements Specification (SRS) Document	Lab	Textbook1: Chapter 6
16	2	Lab 4: Software Requirements Specification (SRS) Document	Lab	Textbook1: Chapter 6
17	1	OOAD Concepts and UML for Agile Projects	Interactive lecture + Whiteboard	Textbook1: Chapter 17
18	2	Code Coverage, Tools, and Agile Design Principles	Interactive lecture + Whiteboard	Textbook1: Chapter 8
19	1	Lab 5: System modeling using UML	Lab	Textbook1: Chapter 17
20	1	Lab 5: System modeling using UML	Lab	Textbook1: Chapter 17
21	5	Agile Testing Principles: Principles of Agile Testers	Interactive lecture + Whiteboard	Textbook1: Chapter 4
22	5	Testing Types: Sanity, Security, Stress, and Performance	Interactive lecture + Whiteboard	Textbook1: Chapter 4
23	5	Lab 6: Writing Test Cases (Sanity, Security, Stress, Performance)	Lab	IEEE-829: Test Plan Outline
24	5	Lab 6: Writing Test Cases (Sanity, Security, Stress, Performance)	Lab	IEEE-829: Test Plan Outline

25	5	Traceability Matrix and Test Planning Standards	Interactive lecture + Whiteboard	Textbook1: Chapter 4
26	5	Test Automation Pyramid and Selenium Introduction	Interactive lecture + Whiteboard	Documentation
27		Lab 7: Test Plan and Traceability Matrix (IEEE Template)	Lab	IEEE-829: Test Plan Outline
28		Lab 7: Test Plan and Traceability Matrix (IEEE Template)	Lab	IEEE-829: Test Plan Outline
29	2	Product Inception: Visioning and Stakeholder Management	Interactive lecture + Whiteboard	Textbook1: Chapter 17
30	3	Sprint Management: Planning, Goals, and Multi-level Planning	Interactive lecture + Whiteboard	Textbook1: Chapter 14
31	3	Lab 8: Scrum Team, Product Backlog, and Sprint Plan	Lab	Textbook1: Chapter 14
32	3	Lab 8: Scrum Team, Product Backlog, and Sprint Plan	Lab	Textbook1: Chapter 14
33	3	Estimation Techniques and Planning Poker	Interactive lecture + Whiteboard	Textbook1: Chapter 7
34	3	Prioritizing Stories: WSJF Technique and Velocity	Interactive lecture + Whiteboard	Textbook1: Chapter 16
35	3	Lab 9: Scrum Team, Product Backlog, and Sprint Plan (Refined)	Lab	Textbook1: Chapter 16
36	3	Lab 9: Scrum Team, Product Backlog, and Sprint Plan (Refined)	Lab	Textbook1: Chapter 16
37	3	Scrum Ceremonies: Daily Stand-up and Progress Tracking	Interactive lecture + Whiteboard	Textbook1: Chapter 20
38	3	Sprint Reviews, Retrospectives, and Burn-down Charts	Interactive lecture + Whiteboard	Textbook1: Chapter 21

39	3	Lab 10: Simulation of Daily Scrum, Planning, Review, Retro	Lab	Textbook1: Chapter 22
40	3	Lab 10: Simulation of Daily Scrum, Planning, Review, Retro	Lab	Textbook1: Chapter 22
41	4	DevOps Foundations: CI/CD, Jenkins, and Automation	Interactive lecture + Whiteboard	Documentation
42	4	Version Control Overview: Branching and Git Workflows	Interactive lecture + Whiteboard	Documentation
43	4	Lab 11: Hands on Git, Github, and GitAction	Lab	Documentation
44	4	Lab 11: Hands on Git, Github, and GitAction	Lab	Documentation
45	4	Containerization: Introduction to Docker and DockerHub	Interactive lecture + Whiteboard	Documentation
46	4	Container Orchestration: Kubernetes and Deployment	Interactive lecture + Whiteboard	Documentation
47	4	Lab 12: Hands on Docker & Docker Hub	Lab	Documentation
48	4	Lab 12: Hands on Docker & Docker Hub	Lab	Documentation
49	4	Code Quality and Testing in DevOps: Pylint and PyUnit	Interactive lecture + Whiteboard	Textbook1: Chapter 20
50	5	Functionality Testing and UI Testing (JUnit/Sonar)	Interactive lecture + Whiteboard	Documentation
51	5	Lab 13: Testing a software project using PyUnit	Lab	IEEE-829: Test Plan Outline
52	5	Lab 13: Testing a software project using PyUnit	Lab	IEEE-829: Test Plan Outline

53	5	Agile Testing Quadrants: Balancing Tests	Interactive lecture + Whiteboard	Documentation
54	5	RPA (UiPath) and A/B Testing in Agile	Interactive lecture + Whiteboard	Documentation
55	5	Lab 14: Create Agile Testing Quadrants (Real-time project)	Lab	IEEE-829: Test Plan Outline
56	5	Lab 14: Create Agile Testing Quadrants (Real-time project)	Lab	IEEE-829: Test Plan Outline
57	4	Managing Microservices and Security (DevSecOps)	Interactive lecture + Whiteboard	Documentation
58	4	End-to-end DevOps Pipelines and Monitoring	Interactive lecture + Whiteboard	Documentation
59	5	Lab 15: End-to-end DevOps demo and project report	Lab	Documentation
60	5	Lab 15: End-to-end DevOps demo and project report	Lab	Documentation

Course Outcomes – Programme Outcomes Matrix

Course Outcomes \ Programme Outcomes		P O 1	P O 2	P O 3	P O 4	P O 5	P O 6	P O 7	P O 8	P O 9	PO 10	PO 11	PO 12
CO1: Evaluate the advantages and disadvantages of Agile development compared to traditional models		3	2	2	–	–	–	–	–	–	–	–	1
CO2: Explore various Agile methodologies such as Scrum, XP, Lean, and Kanban, and provide guidelines based on project needs.		3	3	2	–	–	–	–	–	–	–	1	1
CO3: Create software requirements, design specifications, test plan and Analyze test coverage, requirements traceability for a software project		3	3	3	2	–	–	–	–	–	–	–	1

CO4: Equip students with hands-on experience in using modern DevOps tools (e.g., Git, GitHub, Docker) for automation, and deployment in Agile environments.		3	2	3	3	2	–	–	–	–	–	–	1
--	--	---	---	---	---	---	---	---	---	---	---	---	---

Assessment Plan

Internal Assessment Plan: 70 Marks				
Sl#	Component	Marks	Type of Assessment	Timeline
Continuous Internal Evaluation-1 (20 Marks)				
1	CO1 and CO2	20	Internal Assessment-1 (Pen & Paper)	Week 5
Continuous Internal Evaluation-2 (25 Marks)				
2	CO1 - C04	25	Mid Sem Examination (Theory)	Week 9
Continuous Internal Evaluation-3 (25 Marks)				
3	CO1 – CO4	25	Mini Project Practical Component and Lab Record	Week 14, 15

Mid Sem Assessment Pattern: 25 Marks		
Sl#	Content	
Part A- 5 Marks (Minimum 1 mark, Max 2 marks, Max 5 Questions)		
1	5 Questions of 1 mark each	5
Part B- 20 Marks (Compulsory 4 questions of 5 marks each, there can be max 3 sub divisions)		
2	4 Questions of 5 marks each with 2 or 3 sub divisions possibly	20

Practical Component: 25 Marks		
Sl#	Content	
Part A- 15 Marks		
1	Lab record and activity	15 marks
Part B- 20 Marks		
2	Course Project	35 marks
	Total =(15+35)/2= 25	25 marks

SEE Assessment Pattern: 30 Marks		
Sl#	Content	
Part A - 10 Marks (2 Questions, 5 Marks Each, max 3 Subdivisions)		
1	2 Questions of 5 marks each	10
Part B - 20 Marks (2 Questions, 10 Marks Each, Max Four Subdivisions per Question)		
2	2 Questions of 10 marks each	20

Rubrics for CIE Components

Assessment Component	Type of Component	Rubrics for Assessment
1.	CIE 1	CIE1 will be conducted for 20 marks Part A (10 Marks) - Pen & Paper (Short Ans Type/ In Class Test) Part B (10 Marks) - 5 Marks (MOOC course) - 5 Marks (From exercises/course project of Lab Record) It will be evaluated according to the Scheme of Evaluation prepared by the course faculty
2.	Mid Sem (CIE 2)	Written Test will be conducted for 25 marks and will be evaluated according to the Scheme of Evaluation prepared by the team of course faculty
3.	Practical Component (CIE 3)	CIE3 will be of 25 Marks • Course Project - 20 Marks • DevOps – 20 Marks • Lab Record – 10 Marks (see the details in the Lab Record book) • Total for CIE3 = $(20+20+10)/2 = 25$ Marks It will be evaluated according to the Scheme of Evaluation prepared by the team of course faculty

Semester End Exams (SEE): 30%	
Mode of Exam	Theory
Weightage of exam	30%

1. Detailed Rubrics of Lab Components

Lab Write-up and Execution rubrics (Max: 15 marks)					
SL No	Criteria	Measuring Methods	Excellent	Good	Poor
1	Understanding of problem statements (5 Marks)	Observations	Student exhibits a thorough understanding of requirements for the problem (5 M)	Student has sufficient understanding of the problem statements (3 M)	Student does not have a clear understanding of the problem statements (1 M)
2	Execution (5 Marks)	Observations	Student demonstrates the execution of the program with optimized code and shows the output. Appropriate validations with all the test cases are handled. (5 M)	Student demonstrates the execution of the program without optimization of the code and shows the output. (3 M)	Student has not executed the program or the code fails to demonstrate (1 M)
3	Results and Documentation (5 Marks)	Observations	Documentation with appropriate comments and output is covered in manual. (5 M)	Documentation with only few comments and only few output cases is covered in manual. (3 M)	Documentation with no comments and no output cases <u>is</u> covered in manual. (1 M)

Project Rubrics (Max: 35 marks)					
SL No	Criteria	Measuring Methods	Excellent	Good	Poor
1	Mid Review (15 Marks)	Project	Presents a well-structured and detailed design and architecture, demonstrating a deep understanding of principles. (15 M)	Provides a design and architecture with basic structure, demonstrating some understanding of principles. (12 M)	Lacks a coherent design and architecture or shows minimal understanding of principles. (8 M)
2	Final Review (20 Marks)	Project	Implements the project with clean, efficient, and well-documented code following best practices. (20 M)	Implements the project with functional code that follows some best practices, though may lack efficiency or clarity. (15 M)	Code is poorly implemented, lacking efficiency, clarity, and adherence to best practices. (10 M)

Signature of the Course Faculty